

exp-fips203-mlkem-pke-keygen-k3

FIPS 203 Alg.13 ML-KEM-768 PKE KeyGen 实现方案

ascendc 工程 (ML-KEM-768 W4a / E13)

2026-07-26

摘要

本文档描述 `examples/incubating/ml-kem/ml-kem-768/exp-fips203-mlkem-pke-keygen-k3/`: 在 **Atlas A2 / Ascend910B4** 上以 AscendC 实现 FIPS 203 **Algorithm 13**, 即 ML-KEM-768 ($k=3$) **PKE KeyGen**。

参数锁定: 几何、I/O 与 launch 全部锁定到 `docs/specs/fips203-mlkem768-parameter-card.md` §3.2 D13。**行为基线**: 活跃探针 `ascendc-tests/ml-kem/ml-kem-768/pass-fix-f203-alg13-device-keygen-k3/`。**生产 I/O**: `input/` 仅 `seed_d.bin` 与 NTT LUT; `output/` 仅 `ek_pke.bin` (1184B) 与 `dk_pke.bin` (1152B)。**计划 AI Core 数**: 2 launch; Launch 1 为 AIV_ONLY blockDim=2, Launch 2 为 MIX blockDim=1 (1 AIC + 2 AIV)。**禁止**: 创建 `examples/stable/ml-kem/ml-kem-768/`; 从任何 `frozen/` 目录复制、改写或移植源码; 用 k4 的 `ek/dk` 长度或 16/8 路几何替代 k3 几何。

目录

1	工程边界	3
1.1	允许的代码来源与依赖	3
1.2	禁止项	3
2	数学语义 (Alg.13, $k=3$, $n=256$, $q=3329$)	3
2.1	输入输出	3
2.2	阶段公式	3
3	AscendC 实现规划	4
3.1	Launch 与 AI Core 规模	4
3.2	GM 几何	4
3.3	同步与流水	5
4	AscendC API 表	5
4.1	评估过但未采用	6
5	数学步骤覆盖率	6

目录	2
6 验证计划	7
6.1 默认验收	7
6.2 可选对照	7
7 产出物	7

1 工程边界

1.1 允许的代码来源与依赖

来源	用途
ascendc-tests/ml-kem/ml-kem-768/pass-fix-f203- alg13-device-keygen-k3/ examples/incubating/ml-kem/ml-kem-1024/exp-fips203-mlkem-pke-keygen-k4/ library/shared/ CANN / tikicpulib / CAModel	活跃 D13 k3 行为基线；可复制后自包含适配到本 exp，最终不得保留对探针目录的编译期 include/import 活跃 k4 incubating 结构参考；仅 retarget 到 k3 锁定几何与字节长 SHAKE/Keccak、公共设备侧头文件等共享库 编译、CPU 孪生与 SIM 运行环境

1.2 禁止项

- 禁止从 ascendc-tests/frozen/ 或 examples/frozen/ 拿源码、customspec 或路线。
- 禁止将 a_hat、s_hat、t_hat 等中间态作为默认 I/O 落盘；debug dump 只能显式开关启用。
- 禁止把 k4 的 ek=1568、dk=1536、 $\hat{A}[16]$ 、polyvec8 或零垫 4/8 当作默认。
- 禁止新建或写入 stable-768；本目录只作为 incubating 预研实现。

2 数学语义 (Alg.13, $k = 3$, $n = 256$, $q = 3329$)

2.1 输入输出

张量 / 文件	契约
input/seed_d.bin	4B little-endian uint32, 由 Host 派生 FIPS 203 的 d ；域分离串为 exp-mlkem-f203-2s1e-k3:SEED_D=
input/lut_even_stacked.bin	Not Stage2 LUT；由本目录 golden 脚本生成；不是用户可见交付输入
output/ek_pke.bin	1184B, $\text{ByteEncode}_{12}(\hat{t})$ 的 1152B 后拼接 ρ 的 32B
output/dk_pke.bin	1152B, $\text{ByteEncode}_{12}(\hat{s})$ ，其中 $\hat{s}$ 为前 3 个秘密多项式

2.2 阶段公式

1. $G(d||\text{byte}(3)) \rightarrow (\rho, \sigma)$ ； k 字节锁定为 3。

2. 行 3-7: 对 $(i, j) \in \{0, 1, 2\}^2$ 执行 `SampleNTT`, 生成 $\hat{A} \in R_q^{3 \times 3}$, GM 形状锁定为 `[9, 256] int32`。
3. 行 8-15: 用 $\eta_1 = \eta_2 = 2$ 采样 $s||e$ 共 6 个 poly; GM 形状锁定为 `[6, 256]`。
4. 行 16-20: `polyvec6 NTT` 后计算 $\hat{t} = \hat{A} \circ \hat{s} + \hat{e}$; `InnerProduct` 输出 $P_OUT = 3$, AIV 分片为 2+1。
5. 行 21: 在 `compute launch` 内融合生成 $ek_pke = \text{ByteEncode}_{12}(\hat{t})||\rho$, 同时写 $dk_pke = \text{ByteEncode}_{12}(\hat{s})$ 。

3 AscendC 实现规划

3.1 Launch 与 AI Core 规模

Launch	核类型 / blockDim	数据划分与理由
1 prep	AIV_ONLY, blockDim=2	$\hat{A}$ 共 9 poly 按 5+4 分给两个 AIV; $s e$ 共 6 poly 可先按 B3 交叉分片写 GM, <code>compute</code> 前按 B5 连续 <code>polyvec6</code> 语义消费。选择 2 AIV 是参数卡 §3.2 D13 锁定值, 继承 D13 探针已验同步与 UB 分配。
2 compute	MIX, blockDim=1	1 AIC + 2 AIV; <code>polyvec6 NTT</code> 使用连续 3+3 AIV 映射; <code>InnerProduct</code> 输出 3 行, AIV0 负责 $\hat{t}[0, 1]$, AIV1 负责 $\hat{t}[2]$ 。选择单 MIX AI Core 是 D13 锁定生产档, 避免跨 AI Core GM 交换。

3.2 GM 几何

对象	锁定几何
$\hat{A}$	<code>[9, 256] int32</code> ; 9 个真实 poly, 禁止 pad 到 16
$s e$	<code>[6, 256]</code> ; <code>polyvec6</code> , NTT 前后都按真实 6 poly 管理
NTT S0	<code>[12, 256] int8</code> , 即 <code>[HI6, LO6]</code> ; 每个 AIV 持有完整 poly 的 hi+lo, 禁止 <code>limbsplit</code>
Stage2 mat_c	<code>[48, 128] int32</code> ; 逻辑 $6 \times 4 \times 2$; Cube 的 M 对齐垫到 16 只是硬件对齐, 不表示假 poly
ByteEncode12	$3 \times 384 = 1152\text{B}$; 分别用于 $\hat{t}$ 与 $\hat{s}$

3.3 同步与流水

Prep 内部使用 UB/GM 分段搬运与必要的 PipeBarrier; 两次 launch 之间由 Host 顺序保证 GM 可见性。Compute 内部遵守 B5/B6 已验模式: AIV 写入平面 Stage1/Stage3 GM 后以 CrossCore flag 与 AIC 同步; AIC 写完 mat_c 后 AIV 等待再做 Stage3 merge、InnerProduct 与 ByteEncode12。CPU 仅作为逻辑孪生; SIM 是同步与搬运验收门槛。

4 AscendC API 表

本实现不引入 E13 独有的新 AscendC API; 下表 API 均已有索引记录, 使用约束按 library /documents/CANN-AscendC算子开发接口参考-查阅索引.md 执行。

API 名	分类 / 文档	Atlas A2	本用例 dtype	备注
GetBlockIdx / GetSubBlockIdx	系统变量 / 资源	√	标量索引	区分 AIV 分片与 MIX 子核; 禁止把 CPU 打印误认为多核性能
GlobalTensor / LocalTensor	基础结构	√	uint8/int8/int32	GM/UB 张量契约必须与本 spec 几何一致
DataCopy	Memory 数据搬运	√	uint8/int8/int32	连续块搬运; 长度与地址遵守 32B 对齐约束
PipeBarrier	Pipe 同步	√	—	MTE/Vector/Cube 阶段交界显式同步
CrossCoreSetFlag / CrossCoreWaitFlag	MIX 同步	√	—	AIV/AIC 通过 GM 交接时使用; CPU 过不能删
Mmad / Fixpipe	矩阵计算	√	int8*int8->int32	Stage2 MMAD; 逻辑 M=12, 硬件 M 对齐垫到 16
ShiftRight	矢量算术	√	int32	limb 拆分、mask low bits 与 Barrett 相关右移
Muls / Add / Sub	矢量算术	√	int32/int16	limb、模加减、压缩/编码辅助

Cast	类型转换	√ (受限)	int32/int16/hint8/uint8	遵守 A2 链约束; 不得假设存在 int32->int8 单跳
Gather	矢量搬运/选择	√	int32	仅允许在 Alg.11 / ByteEncode 等非 NTT S1-S3 段按既有路径使用; NTT 三段内禁用
GetValue / SetValue	LocalTensor 标量访问	√	uint8/int32	仅用于 pack 尾部等标量缺口; 不得把 Compares bit 包当逐 lane 布尔

4.1 评估过但未采用

API / 路线	不采用原因
Compares/GatherMask compact 链 高阶 Matmul<>	Alg.13 E13 不需要新增 reject compact; 沿用 D13 已验 Mins/标量 rej 路径, 不新增比较掩码风险 本路线继承 D13 手写 MMAD + 平面 mat_c 契约; 高阶 Matmul 输出布局与 MIX 交接不作为本轮变量
Gather 用于 NTT S1-S3	参数卡与 ML-KEM NTT 定稿禁止; NTT 三段保持平面 mat_c + 连续 poly 批处理

5 数学步骤覆盖率

数学步骤	实现方式 / API	覆盖	缺口说明
$G(d 3)$ 与 XOF	shared Keccak/SHAKE + UB 流水	100% 设备	标量 Keccak 核属本仓 shared; 非 Host oracle
SampleNTT 生成 $\hat{A}[9]$	AIV_ONLY + rej 路径 + DataCopy	100% 设备	5+4 分片; 无零垫
CBD $\eta = 2$ 生成 $s e[6]$	AIV_ONLY + CBD LUT/向量累加	100% 设备	输出按 polyvec6 真实行数
polyvec6 NTT S1-S3	MIX AIV/AIC + Mmad	100% 设备	NTT 内无 Gather

InnerProduct $P = 3$	Alg.11 向量路径 + AIV 2+1	100% 设备	非整除分片显式 处理第 3 行
ByteEncode12 + $ek\ \rho$	ByteEncode12 向量 + 尾部标量 pack	部分向量	pack 尾部标量缺 口沿用 D13/k4 已验实现

6 验证计划

6.1 默认验收

```
cd examples/incubating/ml-kem/ml-kem-768/exp-fips203-mlkem-pke-keygen-k3
bash run.sh -r cpu -v Ascend910B4
SIM_DIRECT=1 bash run.sh -r sim -v Ascend910B4
```

期望: `ek_pke.bin` 为 1184B, `dk_pke.bin` 为 1152B; 若开启 `KEYGEN_VERIFY=1`, 两者均与 golden 对拍 PASS。SIM 通过后记录 Total tick 到 `qa/active_sim_regress_summary.md`, 并检查用例根目录无 `stray core*.dump`、`profile_*log*.toml`。

6.2 可选对照

后续可补 `kat_liboqs_vs_ascendc.sh`, 但 E13 本轮门禁为 CPU + `SIM_DIRECT=1 sim`。liboqs 仅作为黑盒 I/O oracle; 禁止把 liboqs 源码实现同构移植进 AscendC。

7 产出物

- `exp-fips203-mlkem-pke-keygen-k3-实现方案-customspec.tex`
- `exp-fips203-mlkem-pke-keygen-k3-实现方案-customspec.pdf`
- 后续实现文件须与本 `customspec` 同目录自包含, 且自研 Python/C/AscendC 写详细中文注释。